



Strings: A Deeper Look

Chapter 08

8.1 Introduction

- Strings support many of the same sequence operations as lists and tuples
- Strings, like tuples, are immutable
- Here, we take a deeper look at strings
- Introduce regular expressions and the `re` module for matching patterns in text
 - Particularly important in today's data rich applications
- Table below shows many string-processing and NLP-related applications

String and NLP applications

Automated grading of written homework

Chatbots

Compilers and interpreters

Cryptography

Fraud detection

Grammar checkers

Inter-language translation

Search engines

Sentiment analysis

Spam classification

Speech-to-text engines

Text-to-speech engines

Web scraping

Word games

and many more...

8.2 Formatting Strings

8.2.1 Presentation Types

Floating

```
In [1]: f'{17.489:.2f}'
```

```
Out[1]: '17.49'
```

Integer

```
In [2]: f'{10:d}'
```

```
Out[2]: '10'
```

Character

```
In [3]: f'{65:c} {97:c}'
```

```
Out[3]: 'A a'
```

8.2.2 Field Widths and Alignment

- Python **right-aligns numbers** and **left-aligns other values**.
- Python formats `float` values with **six digits of precision**.

```
In [1]: f'[{27:10d}]'
```

```
Out[1]: '[          27]'
```

```
In [2]: f'[{3.5:10f}]'
```

```
Out[2]: '[  3.500000]'
```

```
In [3]: f'[{ "hello":10}]'
```

```
Out[3]: '[hello      ]'
```

Explicitly Specifying Left and Right Alignment in a Field

- Can specify left and right alignment with `<` and `>`:

```
In [4]: f'[{27:<15d}]'
```

```
Out[4]: '[27                ]'
```

```
In [5]: f'[{3.5:<15f}]'
```

```
Out[5]: '[3.500000      ]'
```

```
In [6]: f'[{ "hello":>15}]'
```

```
Out[6]: '[          hello]'
```

Centering a Value in a Field

- Centering attempts to spread the remaining unoccupied character positions equally to the left and right of the formatted value
- Python places the extra space to the right if an odd number of character positions remain

```
In [7]: f'[{27:^7d}]'
```

```
Out[7]: '[ 27  ]'
```

```
In [8]: f'[{3.5:^7f}]'
```

```
Out[8]: '[3.500000]'
```

```
In [9]: f'[{ "hello":^7}]'
```

```
Out[9]: '[ hello ]'
```

8.2.3 Numeric Formatting

Formatting Positive Numbers with Signs

- A `+` before the field width specifies that a positive number should be preceded by a `+`
- To fill the remaining characters of the field with `0`s rather than spaces, place a `0` before the field width (and **after** the `+` if there is one)

```
In [17]: f'[{27:+10d}]'
```

```
Out[17]: '[      +27]'
```

```
In [18]: f'[{27:+010d}]'
```

```
Out[18]: '[+000000027]'
```

Grouping Digits

- Format numbers with **thousands separators** by using a **comma** `,`

```
In [20]: f'{12345678:,d}'
```

```
Out[20]: '12,345,678'
```

```
In [21]: f'{123456.78:,.2f}'
```

```
Out[21]: '123,456.78'
```

8.3 Concatenating and Repeating Strings

- Previously, we used the `+` operator to concatenate strings and the `*` operator to repeat strings
- Also can perform these operations with augmented assignments
- Strings are **immutable**, so each operation assigns a new string object to the variable

```
In [1]: s1 = 'happy'
```

```
In [2]: s2 = 'birthday'
```

```
In [3]: s1 += ' ' + s2
```

```
In [4]: s1
```

```
Out[4]: 'happy birthday'
```

```
In [5]: symbol = '>'
```

```
In [6]: symbol *= 5
```

```
In [7]: symbol
```

```
Out[7]: '>>>>>'
```

8.4 Stripping Whitespace from Strings

Removing Leading and Trailing Whitespace

- `strip` removes leading and trailing whitespace

```
In [34]: sentence = '\t \n This is a test string. \t\t \n'
```

```
In [35]: sentence.strip()
```

```
Out[35]: 'This is a test string.'
```

Removing Leading Whitespace

- `lstrip` removes only leading whitespace

```
In [36]: sentence.lstrip()
```

```
Out[36]: 'This is a test string. \t\t \n'
```

Removing Trailing Whitespace

- `rstrip` removes only trailing whitespace

```
In [37]: sentence.rstrip()
```

```
Out[37]: '\t \n This is a test string.'
```

8.5 Changing Character Case

Capitalizing Only a String's First Character

- Method `capitalize` returns a new string with only the first letter capitalized (sometimes called **sentence capitalization**)

```
In [38]: 'happy birthday'.capitalize()
```

```
Out[38]: 'Happy birthday'
```

Capitalizing the First Character of Every Word in a String

- Method `title` returns a new string with only the first character of each word capitalized (sometimes called **book-title capitalization**)

```
In [39]: 'strings: a deeper look'.title()
```

```
Out[39]: 'Strings: A Deeper Look'
```

8.6 Comparison Operators for Strings

- Strings are compared based on their underlying integer numeric values
- Can check integer codes with `ord`

```
In [40]: print(f'A: {ord("A")}; a: {ord("a")}')
A: 65; a: 97
```

- Compare the strings `'Orange'` and `'orange'` using the comparison operators

```
In [41]: 'Orange' == 'orange'
```

```
Out[41]: False
```

```
In [42]: 'Orange' != 'orange'
```

```
Out[42]: True
```

```
In [43]: 'Orange' <= 'orange'
```

```
Out[43]: True
```

```
In [44]: 'Orange' > 'orange'
```

```
Out[44]: False
```

8.7 Searching for Substrings

- Can search a string for a **substring** to
 - count number of occurrences
 - determine whether a string contains a substring
 - determine the index at which a substring resides in a string
- Each method shown in this section compares characters lexicographically using their underlying numeric values

Counting Occurrences

- `count` returns the number of times its argument occurs in a string

```
In [1]: sentence = 'to be or not to be that is the question'
```

```
In [2]: sentence.count('to')
```

```
Out[2]: 2
```

- If you specify as the second argument a **start index**, `count` searches only the slice `string [ start_index : ]`

```
In [3]: sentence.count('to', 12)
```

```
Out[3]: 1
```

- If you specify as the second and third arguments the **start index** and **end index**, `count` searches only the slice `string [ start_index : end_index ]`

```
In [4]: sentence.count('that', 12, 25)
```

```
Out[4]: 1
```

- Like `count`, the other string methods presented in this section each have **start index** and **end index** arguments

Locating a Substring in a String

- `index` searches for a substring within a string and returns the first index at which the substring is found; otherwise, a `ValueError` occurs:

```
In [5]: sentence.index('be')
```

```
Out[5]: 3
```

- `rindex` performs the same operation as `index`, but searches from the end of the string

```
In [6]: sentence.rindex('be')
```

```
Out[6]: 16
```

- `find` and `rfind` perform the same tasks as `index` and `rindex` but return `-1` if the substring is not found

Determining Whether a String Contains a Substring

- To check whether a string contains a substring, use operator `in` or `not in`

```
In [7]: 'that' in sentence
```

```
Out[7]: True
```

```
In [8]: 'THAT' in sentence
```

```
Out[8]: False
```

```
In [9]: 'THAT' not in sentence
```

```
Out[9]: True
```

Locating a Substring at the Beginning or End of a String

- `startswith` and `endswith` return `True` if the string starts with or ends with a specified substring

```
In [10]: sentence.startswith('to')
```

```
Out[10]: True
```

```
In [11]: sentence.startswith('be')
```

```
Out[11]: False
```

```
In [12]: sentence.endswith('question')
```

```
Out[12]: True
```

```
In [13]: sentence.endswith('quest')
```

```
Out[13]: False
```

8.8 Replacing Substrings

- A common text manipulation is to locate a substring and replace its value
- `replace` searches a string for the substring in its first argument and replaces *each* occurrence with the substring in its second argument
- Can receive an optional third argument specifying the maximum number of replacements

```
In [1]: values = '1\t2\t3\t4\t5'
```

```
In [2]: values.replace('\t', ',')
```

```
Out[2]: '1,2,3,4,5'
```

8.9 Splitting and Joining Strings

- Tokens typically are separated by whitespace characters such as blank, tab and newline, though other characters may be used—the separators are known as **delimiters**

Splitting Strings

- To tokenize a string at a custom delimiter, specify the delimiter string that `split` uses to tokenize the string

```
In [1]: letters = 'A, B, C, D'
```

```
In [2]: letters.split(',')
```

```
Out[2]: ['A', 'B', 'C', 'D']
```

- Specify the maximum number of splits with an integer as the second argument
- Last token is the remainder of the string

```
In [3]: letters.split(',', 2)
```

```
Out[3]: ['A', 'B', 'C, D']
```

- `rsplit` performs the same task as `split` but processes the maximum number of splits from the end of the string toward the beginning

Joining Strings

- `join` concatenates the strings in its argument, which must be an iterable containing only string values
- The separator between the concatenated items is the string on which you call `join`

```
In [4]: letters_list = ['A', 'B', 'C', 'D']
```

```
In [5]: ','.join(letters_list)
```

```
Out[5]: 'A,B,C,D'
```

- Join the results of a list comprehension that creates a list of strings

```
In [6]: ','.join([str(i) for i in range(10)])
```

```
Out[6]: '0,1,2,3,4,5,6,7,8,9'
```

String Methods `partition` and `rpartition`

String method `partition` splits a string into a tuple of three strings based on the method's *separator* argument

- the part of the original string before the separator
- the separator itself
- the part of the string after the separator

```
In [7]: 'Amanda: 89, 97, 92'.partition(': ')
```

```
Out[7]: ('Amanda', ': ', '89, 97, 92')
```

- To search for the separator from the end of the string, use method `rpartition`

```
In [8]: url = 'http://www.deitel.com/books/PyCDS/table_of_contents.html'
```

```
In [9]: rest_of_url, separator, document = url.rpartition('/')
```

```
In [10]: document
```

```
Out[10]: 'table_of_contents.html'
```

```
In [11]: rest_of_url
```

```
Out[11]: 'http://www.deitel.com/books/PyCDS'
```

8.10 Characters and Character-Testing Methods

- In Python, a character is simply a one-character string
- Python provides string methods for testing whether a string matches certain characteristics
- `isdigit` returns `True` if the string on which you call the method contains only the digit characters (0 – 9)
 - Useful for validating data

```
In [1]: '-27'.isdigit()
```

```
Out[1]: False
```

```
In [2]: '27'.isdigit()
```

```
Out[2]: True
```

- `isalnum` returns `True` if the string on which you call the method is alphanumeric (only digits and letters)

```
In [3]: 'A9876'.isalnum()
```

```
Out[3]: True
```

```
In [4]: '123 Main Street'.isalnum()
```

```
Out[4]: False
```

8.10 Characters and Character-Testing Methods (cont.)

- Table of many character-testing methods

String Method	Description
<code>isalnum()</code>	Returns <code>True</code> if the string contains only <i>alphanumeric</i> characters (i.e., digits and letters).
<code>isalpha()</code>	Returns <code>True</code> if the string contains only <i>alphabetic</i> characters (i.e., letters).
<code>isdecimal()</code>	Returns <code>True</code> if the string contains only <i>decimal integer</i> characters (that is, base 10 integers) and does not contain a + or - sign.
<code>isdigit()</code>	Returns <code>True</code> if the string contains only digits (e.g., '0', '1', '2').
<code>isidentifier()</code>	Returns <code>True</code> if the string represents a valid <i>identifier</i> .
<code>islower()</code>	Returns <code>True</code> if all alphabetic characters in the string are <i>lowercase</i> characters (e.g., 'a', 'b', 'c').
<code>isnumeric()</code>	Returns <code>True</code> if the characters in the string represent a <i>numeric value</i> without a + or - sign and without a decimal point.
<code>isspace()</code>	Returns <code>True</code> if the string contains only <i>whitespace</i> characters.
<code>istitle()</code>	Returns <code>True</code> if the first character of each word in the string is the only <i>uppercase</i> character in the word.
<code>isupper()</code>	Returns <code>True</code> if all alphabetic characters in the string are <i>uppercase</i> characters (e.g., 'A', 'B', 'C').

8.12 Introduction to Regular Expressions

- A **regular expression** describes a **search pattern** for **matching** characters in other strings
- Can help you extract data from unstructured text
- Can help you ensure that data is in the correct format before processing it

Validating Data

- Often use regular expressions to **validate the data**
 - A U.S. ZIP Code consists of five digits (such as 02215) or five digits followed by a hyphen and four more digits (such as 02215-4775)
 - A string last name contains only letters, spaces, apostrophes and hyphens
 - An e-mail address contains only the allowed characters in the allowed order
 - A U.S. Social Security number contains three digits, a hyphen, two digits, a hyphen and four digits, and adheres to other rules about the specific numbers that can be used in each group of digits
- Rarely need to create your own regular expressions
- Repositories of existing regular expressions that you can copy and use
 - <https://regex101.com>
 - <http://www.regexlib.com>
 - <https://www.regular-expressions.info>

Other Uses of Regular Expressions

- Extract data from text (sometimes known as **scraping**)
 - e.g., locating all URLs in a web page
 - You might prefer tools like BeautifulSoup, XPath and lxml for this
- Clean data
 - Removing data that's not required, removing duplicate data, handling incomplete data, fixing typos, ensuring consistent data formats, dealing with outliers and more
- Transform data into other formats
 - Reformatting data that was collected as tab-separated or space-separated values into comma-separated values (CSV) for an application that requires data to be in CSV format

8.12.1 re Module and Function fullmatch

- `fullmatch` checks whether the **entire** string in its second argument matches the pattern in its first argument

```
In [26]: import re
```

Matching Literal Characters

```
In [27]: pattern = '02215'
```

```
In [28]: 'Match' if re.fullmatch(pattern, '02215') else 'No match'
```

```
Out[28]: 'Match'
```

```
In [29]: 'Match' if re.fullmatch(pattern, '51220') else 'No match'
```

```
Out[29]: 'No match'
```

- First argument is the regular expression pattern to match
- Second argument is the string that should entirely match the pattern.
- If the second argument matches the pattern in the first argument, `fullmatch` returns an object containing the matching text, which evaluates to `True`
- For no match, `fullmatch` returns `None`, which evaluates to `False`

Metacharacters, Character Classes and Quantifiers

- Regular expressions typically contain various special symbols called **metacharacters**:

Regular expression metacharacters

[] { } () \ * + ^ \$? . |

- `\` **metacharacter** begins each predefined **character class**
- Each matches a specific set of characters

```
In [30]: 'Valid' if re.fullmatch(r'\d{5}', '02215') else 'Invalid'
```

```
Out[30]: 'Valid'
```

```
In [31]: 'Valid' if re.fullmatch(r'\d{5}', '9876') else 'Invalid'
```

```
Out[31]: 'Invalid'
```

- In `\d{5}`, `\d` is a character class representing a digit (0–9)
- A character class is a **regular expression escape sequence** that matches **one** character
- To match more than one, follow the character class with a **quantifier**
- `{5}` repeats `\d` five times to match five consecutive digits

Other Predefined Character Classes

- To match any metacharacter as its **literal** value, precede it by a backslash (`\`)
 - For example, `\\` matches a backslash (`\`) and `\$` matches a dollar sign (`$`)

Character class	Matches
<code>\d</code>	Any digit (0–9).
<code>\D</code>	Any character that is <i>not</i> a digit.
<code>\s</code>	Any whitespace character (such as spaces, tabs and newlines).
<code>\S</code>	Any character that is <i>not</i> a whitespace character.
<code>\w</code>	Any word character (also called an alphanumeric character)—that is, any uppercase or lowercase letter, any digit or an underscore
<code>\W</code>	Any character that is <i>not</i> a word character.

Custom Character Classes

- Square brackets, `[]`, define a **custom character class** that matches a **single** character
- `[aeiou]` matches a lowercase vowel
- `[A-Z]` matches an uppercase letter
- `[a-z]` matches a lowercase letter
- `[a-zA-Z]` matches any lowercase or uppercase letter

```
In [32]: 'Valid' if re.fullmatch('[A-Z][a-z]*', 'Wally') else 'Invalid'
```

```
Out[32]: 'Valid'
```

```
In [33]: 'Valid' if re.fullmatch('[A-Z][a-z]*', 'eva') else 'Invalid'
```

```
Out[33]: 'Invalid'
```

```
In [34]: 'Match' if re.fullmatch('[^a-z]', 'A') else 'No match'
```

```
Out[34]: 'Match'
```

```
In [35]: 'Match' if re.fullmatch('[^a-z]', 'a') else 'No match'
```

```
Out[35]: 'No match'
```

```
In [38]: 'Valid' if re.fullmatch('[A-Z][a-z]+', 'Wally') else 'Invalid'
```

```
Out[38]: 'Valid'
```

```
In [39]: 'Valid' if re.fullmatch('[A-Z][a-z]+', 'E') else 'Invalid'
```

```
Out[39]: 'Invalid'
```

- The * quantifier matches zero or more occurrences of the subexpression to its left.
- The + quantifier matches at least one occurrence of the subexpression to its left.
- The ^ quantifier matches any character that's not specified.

Other Quantifiers

- Can match **between n and m (inclusive) occurrences** of a subexpression with the `{n,m}` quantifier

```
In [46]: 'Match' if re.fullmatch(r'\d{3,6}', '123') else 'No match'
```

```
Out[46]: 'Match'
```

```
In [47]: 'Match' if re.fullmatch(r'\d{3,6}', '123456') else 'No match'
```

```
Out[47]: 'Match'
```

```
In [48]: 'Match' if re.fullmatch(r'\d{3,6}', '1234567') else 'No match'
```

```
Out[48]: 'No match'
```

```
In [49]: 'Match' if re.fullmatch(r'\d{3,6}', '12') else 'No match'
```

```
Out[49]: 'No match'
```

8.12.2 Replacing Substrings and Splitting Strings

- `sub` function replaces patterns in a string
- `split` function breaks a string into pieces, based on patterns

Function `sub`—Replacing Patterns

- `sub` function replaces **all** occurrences of a pattern with the replacement text you specify

```
In [109]: import re
```

```
In [110]: re.sub(r'\t', ' ', '1\t2\t3\t4')
```

```
Out[110]: '1, 2, 3, 4'
```

- Three required arguments:
 - the *pattern to match* (the tab character `'\t'`)
 - the *replacement text* (`' '`) and
 - the *string to be searched* (`'1\t2\t3\t4'`)

- Keyword argument `count` can be used to specify the maximum number of replacements

```
In [111]: re.sub(r'\t', ' ', '1\t2\t3\t4', count=2)
```

```
Out[111]: '1, 2, 3\t4'
```

Function `split`

- `split` function *tokenizes* a string, using a regular expression to specify the *delimiter*
- Returns a list of strings

```
In [112]: re.split(r'\s*', '1, 2, 3,4, 5,6,7,8')
```

```
Out[112]: ['1', '2', '3', '4', '5', '6', '7', '8']
```

- Keyword argument `maxsplit` specifies maximum number of splits

```
In [113]: re.split(r'\s*', '1, 2, 3,4, 5,6,7,8', maxsplit=3)
```

```
Out[113]: ['1', '2', '3', '4, 5,6,7,8']
```

8.12.3 Other Search Functions; Accessing Matches

Function `search`—Finding the First Match Anywhere in a String

- `search` looks in a string for the *first* occurrence of a substring that matches a regular expression and returns a **match object** (of type `SRE_Match`) that contains the matching substring
- Match object's `group` method returns that substring

```
In [114]: import re
```

```
In [115]: result = re.search('Python', 'Python is fun')
```

```
In [116]: result.group() if result else 'not found'
```

```
Out[116]: 'Python'
```

- `search` returns `None` if the string does *not* contain the pattern

```
In [117]: result2 = re.search('fun!', 'Python is fun')
```

```
In [118]: result2.group() if result2 else 'not found'
```

```
Out[118]: 'not found'
```

```
32]: result3 = re.search(r'\d{4}', 'Python is fun since 1991')
```

```
33]: result3.group() if result3 else 'not found'
```

```
33]: '1991'
```

- Can search for a match only at the *beginning* of a string with function `match`

```
[28]: result4 = re.match('Python', 'Python is fun since 1994')
```

```
[29]: result4.group() if result4 else 'not found'
```

```
[29]: 'Python'
```

```
[30]: result5 = re.match('fun', 'Python is fun since 1994')
```

```
[31]: result5.group() if result5 else 'not found'
```

```
[31]: 'not found'
```

Ignoring Case with the Optional `flags` Keyword Argument

- Many `re` module functions receive an optional `flags` keyword argument
- Changes how regular expressions are matched

```
In [119]: result3 = re.search('Sam', 'SAM WHITE', flags=re.IGNORECASE)
```

```
In [120]: result3.group() if result3 else 'not found'
```

```
Out[120]: 'SAM'
```

Metacharacters That Restrict Matches to the Beginning or End of a String

- `^` **metacharacter** at the beginning of a regular expression (and not inside square brackets) is an anchor
- Indicates that the expression matches only the *beginning* of a string

```
In [121]: result = re.search('^Python', 'Python is fun')
```

```
In [122]: result.group() if result else 'not found'
```

```
Out[122]: 'Python'
```

```
In [123]: result = re.search('^fun', 'Python is fun')
```

```
In [124]: result.group() if result else 'not found'
```

```
Out[124]: 'not found'
```

- **\$ metacharacter** at the end of a regular expression is an anchor indicating that the expression matches only the *end* of a string

```
In [125]: result = re.search('Python$', 'Python is fun')
```

```
In [126]: result.group() if result else 'not found'
```

```
Out[126]: 'not found'
```

```
In [127]: result = re.search('fun$', 'Python is fun')
```

```
In [128]: result.group() if result else 'not found'
```

```
Out[128]: 'fun'
```

Function `findall` —Finding All Matches in a String

- **`findall`** finds *every* matching substring in a string
- Returns a list of the matching substrings

```
In [129]: contact = 'Wally White, Home: 555-555-1234, Work: 555-555-4321'
```

```
In [130]: re.findall(r'\d{3}-\d{3}-\d{4}', contact)
```

```
Out[130]: ['555-555-1234', '555-555-4321']
```

Capturing Substrings in a Match

- Use **parentheses metacharacters**—(and)—to capture substrings in a match

```
In [132]: text = 'Charlie Cyan, e-mail: demo1@deitel.com'
```

```
In [133]: pattern = r'([A-Z][a-z]+ [A-Z][a-z]+), e-mail: (\w+@\w+\.\w{3})'
```

```
In [134]: result = re.search(pattern, text)
```

- The regular expression specifies two substrings to capture, each denoted by the metacharacters (and)
- (and) do *not* affect whether the `pattern` is found in the string `text`
- `match` function returns a match object *only* if the *entire* `pattern` is found in the string `text`
- `match` object's `groups` method returns a tuple of the captured substrings

```
In [135]: result.groups()
```

```
Out[135]: ('Charlie Cyan', 'demo1@deitel.com')
```

8.13 Intro to Data Science: Pandas, Regular Expressions and Data Munging

- Data does not always come in forms ready for analysis
- Data could be
 - wrong format
 - incorrect
 - missing
- Data scientists can spend as much as 75% of their time preparing data before they begin their studies
- Called **data munging** or **data wrangling**
- Two of the most important steps in data munging are *data cleaning* and *transforming data* into optimal formats for database systems and analytics software
- Common data cleaning examples include:
 - deleting observations with missing values,
 - substituting reasonable values for missing values,
 - deleting observations with bad values,
 - substituting reasonable values for bad values,
 - tossing outliers (although sometimes you'll want to keep them),
 - duplicate elimination (although sometimes duplicates are valid),
 - dealing with inconsistent data,
 - and more.

8.13 Intro to Data Science: Pandas, Regular Expressions and Data Munging (cont.)

- Data cleaning is a difficult and messy process where you could easily make bad decisions that would negatively impact your results
- The actions data scientists take can vary per project, be based on the quality and nature of the data and be affected by evolving organization and professional standards
- Common data transformations include:
 - removing unnecessary data and *features* (we'll say more about features in the data science case studies),
 - combining related features,
 - sampling data to obtain a representative subset (we'll see in the data science case studies that *random sampling* is particularly effective for this and we'll say why),
 - standardizing data formats,
 - grouping data,
 - and more.

Cleaning Your Data

- Bad data values and missing values can significantly impact data analysis
- Some data scientists advise against any attempts to insert “reasonable values”
 - Instead, they advocate clearly marking missing data and leaving it up to the data analytics package to handle the issue
- Consider a hospital that records patients’ temperatures (and probably other vital signs) four times per day
- Assume that the data consists of a name and four `float` values, such as

```
['Brown, Sue', 98.6, 98.4, 98.7, 0.0]
```

- Patient’s first three recorded temperatures are 99.7, 98.4 and 98.7
- Last temperature was missing and recorded as 0.0, perhaps because the sensor malfunctioned
- Average of the first three values is 98.57, which is close to normal
- If you calculate the average temperature *including* the missing value for which 0.0 was substituted, the average is only 73.93, clearly a questionable result
- Crucial to “get the data right.”
- One way to clean the data is to substitute a *reasonable* value for the missing temperature, such as the average of the patient’s other readings

Data Validation

- `Series` of five-digit ZIP Codes from a dictionary of city-name/five-digit-ZIP-Code key–value pairs
- Intentionally entered an invalid ZIP Code for Miami

```
In [1]: import pandas as pd
```

```
In [2]: zips = pd.Series({'Boston': '02215', 'Miami': '3310'})
```

```
In [3]: zips
```

```
Out[3]: Boston    02215  
Miami         3310  
dtype: object
```

- The “second column” represents the `Series`’ ZIP Code *values* (from the dictionary’s values)
- The “first column” represents their *indices* (from the dictionary’s keys)
- Can use regular expressions with Pandas to validate data
- The `str` **attribute** of a `Series` provides string-processing and various regular expression methods
- Use the `str` attribute’s `match` **method** to check whether each ZIP Code is valid:

Data Validation

```
In [4]: zips.str.match(r'\d{5}')
```

```
Out[4]: Boston      True  
Miami      False  
dtype: bool
```

- `match` applies the regular expression `\d{5}` to *each* `Series` element
- Returns a new `Series` containing `True` for each valid element
- Several ways to deal with invalid data
- One is to catch it at its source and interact with the source to correct the value
 - Not always possible
- In the case of the bad Miami ZIP Code of `3310`, we might look for Miami ZIP Codes beginning with `3310`
 - There are two—`33101` and `33109`
 - We could pick one of those

- Sometimes, rather than matching an *entire* value to a pattern, you'll want to know whether a value contains a *substring* that matches the pattern
- Use method **contains** instead of **match**

```
In [5]: cities = pd.Series(['Boston, MA 02215', 'Miami, FL 33101'])
```

```
In [6]: cities
```

```
Out[6]: 0    Boston, MA 02215  
        1    Miami, FL 33101  
        dtype: object
```

```
In [7]: cities.str.contains(r' [A-Z]{2} ')
```

```
Out[7]: 0    True  
        1    True  
        dtype: bool
```

```
In [8]: cities.str.match(r' [A-Z]{2} ')
```

```
Out[8]: 0    False  
        1    False  
        dtype: bool
```

Reformatting Your Data

- Consider munging data into a different format
- Assume that an application requires U.S. phone numbers in the format ###-###-####
- The phone numbers have been provided to us as 10-digit strings without hyphens

```
In [9]: contacts = [['Mike Green', 'demo1@deitel.com', '5555555555'],  
                    ['Sue Brown', 'demo2@deitel.com', '5555551234']]
```

```
In [10]: contactsdf = pd.DataFrame(contacts,  
                                   columns=['Name', 'Email', 'Phone'])
```

```
In [11]: contactsdf
```

Out[11]:

	Name	Email	Phone
0	Mike Green	demo1@deitel.com	5555555555
1	Sue Brown	demo2@deitel.com	5555551234

Reformatting Your Data

- Munge the data with functional-style programming
- Can *map* the phone numbers to the proper format by calling the `Series` method `map` on the `DataFrame`'s `'Phone'` column
- `map`'s argument is a *function* that receives a value and returns the *mapped* value
- Our function `get_formatted_phone` maps 10 consecutive digits into the format `###-###-####`

```
In [12]: import re
```

```
In [13]: def get_formatted_phone(value):  
    result = re.fullmatch(r'(\d{3})(\d{3})(\d{4})', value)  
    return '-'.join(result.groups()) if result else value
```

- Regular expression in the block's first statement matches *only* 10 consecutive digits
- Captures substrings containing the first three digits, the next three digits and the last four digits
- `return` statement:
 - If `result` is `None`, returns `value` unmodified
 - Otherwise, calls `result.groups()` to get a tuple containing the captured substrings and pass that tuple to string method `join` to concatenate the elements, separating each from the next with `'-'` to form the mapped phone number

- `Series` method `map` returns a new `Series` containing the results of calling its function argument for each value in the column

```
In [14]: formatted_phone = contactsdf['Phone'].map(get_formatted_phone)
```

```
In [15]: formatted_phone
```

```
Out[15]: 0    555-555-5555  
1    555-555-1234  
Name: Phone, dtype: object
```

- Once you've confirmed that the data is in the correct format, you can update it in the original `DataFrame`

```
In [16]: contactsdf['Phone'] = formatted_phone
```

```
In [17]: contactsdf
```

```
Out[17]:
```

	Name	Email	Phone
0	Mike Green	demo1@deitel.com	555-555-5555
1	Sue Brown	demo2@deitel.com	555-555-1234